1. ¿Por qué MapReduce es considerado un modelo escalable de procesamiento distribuido? (10 pts.)

MapReduce es escalable porque divide el procesamiento en dos fases (Map y Reduce) que pueden ejecutarse en paralelo sobre múltiples nodos de un clúster. Al distribuir los datos y las tareas, el sistema puede manejar volúmenes crecientes sin perder eficiencia. Además, permite añadir más nodos para aumentar capacidad sin modificar el código base.

2. ¿Qué problema ocurre cuando un mapper demora mucho más que el resto y cómo afecta al rendimiento total? (10 pts.)

Este fenómeno se conoce como straggler. Si un mapper tarda mucho más, el resto de las tareas deben esperar a que termine, retrasando la etapa Shuffle y, por ende, todo el flujo. El rendimiento total se ve afectado porque el tiempo de ejecución queda determinado por el nodo más lento. Una solución es aplicar técnicas de speculative execution, donde se duplican tareas lentas en otros nodos.

3. ¿Por qué la etapa Shuffle/Sort suele ser la más costosa en MapReduce? (10 pts.)

Shuffle/Sort implica mover grandes volúmenes de datos entre nodos a través de la red, agrupar claves iguales y ordenarlas. Este proceso consume ancho de banda, memoria y CPU, por lo que suele ser el cuello de botella. La complejidad aumenta con datasets grandes, ya que cada clave debe ser enviada al reducer correspondiente.

4. Compare procesamiento secuencial versus distribuido para un dataset de gran tamaño. (10 pts.)

Secuencial: todo el dataset se procesa en un solo equipo, lo que limita el rendimiento y puede saturar memoria y CPU. El tiempo de ejecución crece de manera lineal con el tamaño de los datos.

Distribuido: el dataset se divide en fragmentos y se procesa en paralelo en múltiples nodos. Esto reduce tiempos de ejecución y permite escalar horizontalmente. La desventaja es la complejidad de coordinación y el costo de comunicación entre nodos.

5. ¿Qué ocurre si un reducer recibe muchos más datos que los demás y cómo podría solucionarse? (10 pts.)

Se produce un desbalance de carga: ese reducer se convierte en un cuello de botella, retrasando el resultado final. Para solucionarlo se puede:

Usar particionadores balanceados que distribuyan las claves de manera uniforme.

Implementar técnicas de load balancing dinámico.

Ajustar la lógica de particionamiento para evitar que una sola clave concentre demasiados datos.